

Module 4: Functions Part 1 - Text & Numeric

 **SQL Server Version:** This module has been converted from MySQL to SQL Server (T-SQL) syntax.

Functions are pre-built operations that perform specific tasks on data. They allow us to transform, manipulate, and analyze data directly within our SQL queries. This module covers two fundamental categories of functions: string (text) functions and numeric (mathematical) functions. These functions enable us to clean data, extract information, perform calculations, and format output according to our needs.

1. Understanding SQL Functions

SQL functions accept input values (called arguments or parameters), process them, and return a result. Functions can be used in SELECT TOP 5 statements, WHERE clauses, and other parts of SQL queries.

1.1 Function Syntax

```
FUNCTION_NAME(argument1, argument2, ...)
```

Functions can take zero, one, or multiple arguments depending on their purpose. The result is returned immediately and can be used in calculations or displayed in query results.

1.2 Types of Functions Covered in This Module

- **String Functions** - Manipulate and transform text data
 - **Numeric Functions** - Perform mathematical operations on numbers
-

2. String Functions

String functions operate on text data, allowing us to concatenate, extract, replace, and transform character strings. These functions are essential for data cleaning, formatting, and text processing.

3. String Concatenation (+ Operator)

In SQL Server, strings are combined using the + operator instead of the CONCAT function. This is useful for creating full names from first and last names, building addresses, or generating formatted output.

3.1 Basic String Concatenation Syntax

```
string1 + string2 + string3 + ...
```

3.2 Simple Concatenation Example

```
SELECT 'Hello' + ' ' + 'World';
```

Output:

```
+-----+
| String Concatenation('Hello', ' ', 'World') |
+-----+
| Hello World                                |
+-----+
1 row in set (0.00 sec)
```

This combines three strings: 'Hello', a space ' ', and 'World' into a single string 'Hello World'.

3.3 Concatenation with Table Data

```
SELECT sname + ' (Roll: ' + roll + ' )' AS student_info
FROM students
```

Output:

```
+-----+
| student_info |
+-----+
| Aarav (Roll: 1) |
| Vivaan (Roll: 2) |
| Aditya (Roll: 3) |
| Vihaan (Roll: 4) |
| Arjun (Roll: 5) |
+-----+
5 rows in set (0.01 sec)
```

This creates a formatted string combining the student name with their roll number. The AS keyword creates an alias for the column.

3.4 Creating Full Names

A common use case is combining first and last names:

```
SELECT TOP 5 first_name + ' ' + last_name AS full_name
FROM employees;
```

Output:

```
+-----+
| full_name |
+-----+
| Rajesh Kumar |
| Priya Sharma |
| Amit Patel |
| Sneha Gupta |
+-----+
4 rows in set (0.00 sec)
```

4. SUBSTRING Function

The SUBSTRING function extracts a portion of a string based on a starting position and optional length. This is useful for extracting parts of codes, phone numbers, or any fixed-format data.

4.1 SUBSTRING Syntax

```
SUBSTRING(string, start_position, length)
```

The start_position begins at 1 (not 0). If length is omitted, the function returns everything from the start position to the end of the string.

4.2 Basic SUBSTRING Examples

```
SELECT SUBSTRING('Hello World', 1, 5);
```

Output:

```
+-----+
| SUBSTRING('Hello World', 1, 5) |
+-----+
| Hello                           |
+-----+
1 row in set (0.00 sec)
```

This extracts 5 characters starting from position 1, returning 'Hello'.

```
SELECT SUBSTRING('Hello World', 7);
```

Output:

```
+-----+
| SUBSTRING('Hello World', 7) |
+-----+
| World                        |
+-----+
1 row in set (0.00 sec)
```

Without a length parameter, it returns everything from position 7 to the end.

4.3 SUBSTRING with Table Data

```
SELECT sname, SUBSTRING(sname, 1, 3) AS name_prefix
FROM students
WHERE sname LIKE 'A%'
```

Output:

```
+-----+-----+
| sname  | name_prefix |
+-----+-----+
| Aarav  | Aar         |
| Aditya | Adi         |
| Arjun  | Arj         |
| Advika | Adv         |
| Ananya | Ana         |
+-----+-----+
5 rows in set (0.01 sec)
```

This extracts the first three characters of each student name starting with 'A'.

4.4 Extracting Area Codes

```
SELECT TOP 5 phone_number,  
             SUBSTRING(phone_number, 1, 3) AS area_code  
FROM customers;
```

Output:

```
+-----+-----+  
| phone_number | area_code |  
+-----+-----+  
| 9876543210  | 987      |  
| 8765432109  | 876      |  
| 7654321098  | 765      |  
+-----+-----+  
3 rows in set (0.00 sec)
```

5. TRIM, LTRIM, and RTRIM Functions

These functions remove unwanted spaces from strings. They are essential for data cleaning, especially when dealing with imported data that may have inconsistent spacing.

5.1 TRIM Function

TRIM removes spaces from both the beginning and end of a string:

```
SELECT TRIM(' Hello World ');
```

Output:

```
+-----+  
| TRIM(' Hello World ') |  
+-----+
```

```
| Hello World |  
+-----+  
1 row in set (0.00 sec)
```

5.2 LTRIM Function

LTRIM removes spaces only from the left (beginning) of a string:

```
SELECT LTRIM(' Hello World ');
```

Output:

```
+-----+  
| LTRIM(' Hello World ') |  
+-----+  
| Hello World |  
+-----+  
1 row in set (0.00 sec)
```

Notice the spaces on the right are preserved.

5.3 RTRIM Function

RTRIM removes spaces only from the right (end) of a string:

```
SELECT RTRIM(' Hello World ');
```

Output:

```
+-----+  
| RTRIM(' Hello World ') |  
+-----+
```

```
|      Hello World      |
+-----+
1 row in set (0.00 sec)
```

The leading spaces remain intact.

5.4 Practical Use - Cleaning Data

```
SELECT sname, TRIM(sname) AS cleaned_name
FROM students
WHERE sname LIKE '% '
```

Output:

```
+-----+-----+
| sname  | cleaned_name |
+-----+-----+
| Aarav  | Aarav       |
| Priya  | Priya       |
| Rahul  | Rahul       |
| Sneha  | Sneha       |
| Amit   | Amit        |
+-----+-----+
5 rows in set (0.01 sec)
```

This removes trailing spaces from names that have them.

6. UPPER and LOWER Functions

These functions convert strings to uppercase or lowercase, which is useful for standardizing data or performing case-insensitive comparisons.

6.1 UPPER Function

```
SELECT TOP 5 UPPER('Hello World');
```

Output:

```
+-----+  
| UPPER('Hello World') |  
+-----+  
| HELLO WORLD          |  
+-----+  
1 row in set (0.00 sec)
```

6.2 LOWER Function

```
SELECT LOWER('Hello World');
```

Output:

```
+-----+  
| LOWER('Hello World') |  
+-----+  
| hello world          |  
+-----+  
1 row in set (0.00 sec)
```

6.3 Standardizing Data

```
SELECT sname,
       UPPER(sname) AS uppercase_name,
       LOWER(sname) AS lowercase_name
FROM students
```

Output:

```
+-----+-----+-----+
| sname | uppercase_name | lowercase_name |
+-----+-----+-----+
| Aarav | AARAV          | aarav          |
| Vivaan | VIVAAN         | vivaan         |
| Aditya | ADITYA         | aditya         |
| Vihaan | VIHAAN         | vihaan         |
| Arjun  | ARJUN          | arjun          |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

6.4 Case-Insensitive Search

```
SELECT TOP 5 * FROM students
WHERE LOWER(sname) = LOWER('AARAV');
```

Output:

```
+-----+-----+-----+-----+-----+-----+-----+
| roll | sname | english | maths | science | standard | division |
+-----+-----+-----+-----+-----+-----+-----+
| 1    | Aarav | 78      | 85     | 82      | 10       | A        |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

This finds the student regardless of how the name is capitalized in the database.

7. REPLACE Function

The REPLACE function searches for a substring within a string and replaces all occurrences with another substring. This is useful for correcting data errors or standardizing formats.

7.1 REPLACE Syntax

```
REPLACE(string, search_string, replacement_string)
```

7.2 Basic REPLACE Example

```
SELECT REPLACE('Hello World', 'World', 'SQL');
```

Output:

```
+-----+
| REPLACE('Hello World', 'World', 'SQL') |
+-----+
| Hello SQL                               |
+-----+
1 row in set (0.00 sec)
```

7.3 Replacing Multiple Occurrences

```
SELECT REPLACE('banana', 'a', 'o');
```

Output:

```

+-----+
| REPLACE('banana', 'a', 'o') |
+-----+
| bonono                        |
+-----+
1 row in set (0.00 sec)

```

All occurrences of 'a' are replaced with 'o'.

7.4 Practical Use - Data Correction

```

SELECT address,
       REPLACE(address, 'Pune', 'Pune City') AS updated_address
FROM students
WHERE address = 'Pune'

```

Output:

```

+-----+-----+
| address | updated_address |
+-----+-----+
| Pune    | Pune City      |
| Pune    | Pune City      |
| Pune    | Pune City      |
| Pune    | Pune City      |
| Pune    | Pune City      |
+-----+-----+
5 rows in set (0.01 sec)

```

7.5 Removing Characters

You can remove characters by replacing them with an empty string:

```
SELECT TOP 5 REPLACE('123-456-7890', '-', '');
```

Output:

```
+-----+
| REPLACE('123-456-7890', '-', '') |
+-----+
| 1234567890                        |
+-----+
1 row in set (0.00 sec)
```

8. LOCATE and CHARINDEX Functions

These functions find the position of a substring within a string. LOCATE is used in MySQL, while CHARINDEX is used in SQL Server. They return the position (starting from 1) where the substring is found, or 0 if not found.

8.1 LOCATE Syntax (MySQL)

```
LOCATE(substring, string, start_position)
```

The start_position is optional and specifies where to begin the search.

8.2 Basic LOCATE Example

```
SELECT LOCATE('World', 'Hello World');
```

Output:

```

+-----+
| LOCATE('World', 'Hello World') |
+-----+
|              7              |
+-----+
1 row in set (0.00 sec)

```

'World' starts at position 7 in the string 'Hello World'.

8.3 LOCATE with Start Position

```
SELECT LOCATE('p', 'pineapple');
```

Output:

```

+-----+
| LOCATE('p', 'pineapple') |
+-----+
|              1              |
+-----+
1 row in set (0.00 sec)

```

```
SELECT LOCATE('p', 'pineapple', 2);
```

Output:

```

+-----+
| LOCATE('p', 'pineapple', 2) |
+-----+

```

```

|          7          |
+-----+
1 row in set (0.00 sec)

```

The first query finds 'p' at position 1. The second query starts searching from position 2, finding the next 'p' at position 7.

```
SELECT LOCATE('p', 'pineapple', 7);
```

Output:

```

+-----+
| LOCATE('p', 'pineapple', 7) |
+-----+
|          7          |
+-----+
1 row in set (0.00 sec)

```

8.4 Checking if Substring Exists

```

SELECT sname,
       LOCATE('ar', LOWER(sname)) AS position
FROM students
WHERE LOCATE('ar', LOWER(sname)) > 0

```

Output:

```

+-----+-----+
| sname  | position |
+-----+-----+
| Aarav  | 3        |
| Arnav  | 1        |
| Aarna  | 2        |

```

```
| Arjun | 1 |
| Kartik | 2 |
+-----+-----+
5 rows in set (0.01 sec)
```

This finds students whose names contain 'ar' and shows the position where it occurs.

9. LENGTH Function

The LENGTH function returns the number of characters in a string. This is useful for validation, finding names of certain lengths, or analyzing data patterns.

9.1 Basic LENGTH Example

```
SELECT TOP 5 LEN('Hello World');
```

Output:

```
+-----+
| LEN('Hello World') |
+-----+
|          11         |
+-----+
1 row in set (0.00 sec)
```

The string 'Hello World' contains 11 characters (including the space).

9.2 Finding Names by Length

```
SELECT sname, LEN(sname) AS name_length
FROM students
WHERE LEN(sname) >= 7
```

Output:

sname	name_length
Dhruvansh	9
Krishnam	8
Shaurya	7
Aaradhya	8
Shivansh	8

5 rows in set (0.01 sec)

9.3 Data Validation

```
SELECT TOP 5 email, LEN(email) AS email_length
FROM employees
WHERE LEN(email) < 15;
```

Output:

email	email_length
	13

```
| [email protected] | 12 |
+-----+-----+
2 rows in set (0.00 sec)
```

This identifies emails that might be too short or potentially invalid.

10. Combining String Functions

String functions can be combined to perform complex transformations in a single query.

10.1 Creating Formatted Email Addresses

```
SELECT LOWER(first_name, '.', LOWER(last_name), '@company.com') AS email
FROM employees;
```

Output:

```
+-----+
| email |
+-----+
| [email protected] |
| [email protected] |
| [email protected] |
| [email protected] |
+-----+
4 rows in set (0.00 sec)
```

10.2 Extracting and Formatting

```
SELECT sname,  
       UPPER(SUBSTRING(sname, 1, 1)) AS first_letter,  
       UPPER(SUBSTRING(sname + 1 + 1),  
                 LOWER(SUBSTRING(sname, 2))) AS formatted_name  
FROM students
```

Output:

sname	first_letter	formatted_name
AARAV	A	Aarav
VIVAAN	V	Vivaan
ADITYA	A	Aditya
VIHAAN	V	Vihaan
ARJUN	A	Arjun

5 rows in set (0.01 sec)

This converts names to proper case (first letter uppercase, rest lowercase) even if they're stored in all caps.

11. Numeric Functions

Numeric functions perform mathematical operations on numbers. They are essential for calculations, rounding, and statistical analysis within SQL queries.

12. ROUND Function

The ROUND function rounds a number to a specified number of decimal places. This is crucial for financial calculations and displaying clean numeric output.

12.1 ROUND Syntax

```
ROUND(number, decimal_places)
```

If decimal_places is omitted, the number is rounded to the nearest integer.

12.2 Basic ROUND Examples

```
SELECT TOP 5 ROUND(123.456);
```

Output:

```
+-----+  
| ROUND(123.456) |  
+-----+  
|          123   |  
+-----+  
1 row in set (0.00 sec)
```

```
SELECT ROUND(123.456, 2);
```

Output:

```
+-----+  
| ROUND(123.456, 2) |  
+-----+  
|          123.46   |  
+-----+  
1 row in set (0.00 sec)
```

```
SELECT ROUND(123.456, 1);
```

Output:

```
+-----+
| ROUND(123.456, 1) |
+-----+
|          123.5     |
+-----+
1 row in set (0.00 sec)
```

12.3 ROUND with Table Data

```
SELECT sname,
       (english + maths + science) / 3 AS exact_average,
       ROUND((english + maths + science) / 3, 2) AS rounded_average
FROM students
```

Output:

```
+-----+-----+-----+
| sname   | exact_average | rounded_average |
+-----+-----+-----+
| Aarav   | 81.666667    | 81.67          |
| Vivaan  | 76.333333    | 76.33          |
| Aditya  | 82.000000    | 82.00          |
| Vihaan  | 73.333333    | 73.33          |
| Arjun   | 79.666667    | 79.67          |
+-----+-----+-----+
5 rows in set (0.01 sec)
```

12.4 Rounding to Nearest 10

```
SELECT TOP 5 esalary,  
       ROUND(esalary, -2) AS rounded_to_hundred  
FROM employees;
```

Output:

esalary	rounded_to_hundred
45000.00	45000
52500.00	52500
48750.00	48800
31200.00	31200

4 rows in set (0.00 sec)

Using negative decimal places rounds to the left of the decimal point (tens, hundreds, etc.).

13. FLOOR Function

The FLOOR function returns the largest integer value that is less than or equal to the number. It always rounds down.

13.1 Basic FLOOR Examples

```
SELECT FLOOR(123.456);
```

Output:

```

+-----+
| FLOOR(123.456) |
+-----+
|      123      |
+-----+
1 row in set (0.00 sec)

```

```
SELECT FLOOR(123.999);
```

Output:

```

+-----+
| FLOOR(123.999) |
+-----+
|      123      |
+-----+
1 row in set (0.00 sec)

```

Even though 123.999 is very close to 124, FLOOR rounds down to 123.

13.2 FLOOR with Negative Numbers

```
SELECT FLOOR(-123.456);
```

Output:

```

+-----+
| FLOOR(-123.456) |
+-----+

```

```
|      -124      |
+-----+
1 row in set (0.00 sec)
```

For negative numbers, FLOOR returns the next lower integer (more negative).

13.3 Practical Use - Age Calculation

```
SELECT sname,
       dob,
       FLOOR(DATEDIFF(day, dob, CAST(GETDATE() AS DATE)) / 365.25) AS age_in_years
FROM students
```

Output:

```
+-----+-----+-----+
| sname  | dob      | age_in_years |
+-----+-----+-----+
| Aarav  | 2010-03-15 | 15          |
| Vivaan | 2010-07-22 | 15          |
| Aditya | 2009-11-08 | 16          |
| Vihaan | 2010-01-30 | 15          |
| Arjun  | 2010-09-12 | 15          |
+-----+-----+-----+
5 rows in set (0.01 sec)
```

FLOOR ensures we get the completed years of age, not a decimal.

14. CEILING (or CEIL) Function

The CEILING function returns the smallest integer value that is greater than or equal to the number. It always rounds up.

14.1 Basic CEILING Examples

```
SELECT TOP 5 CEILING(123.456);
```

Output:

```
+-----+
| CEILING(123.456) |
+-----+
|          124     |
+-----+
1 row in set (0.00 sec)
```

```
SELECT CEILING(123.001);
```

Output:

```
+-----+
| CEILING(123.001) |
+-----+
|          124     |
+-----+
1 row in set (0.00 sec)
```

Even a small fraction causes CEILING to round up to the next integer.

14.2 CEILING with Negative Numbers

```
SELECT CEILING(-123.456);
```

Output:

```

+-----+
| CEILING(-123.456) |
+-----+
|          -123     |
+-----+
1 row in set (0.00 sec)

```

For negative numbers, CEILING rounds toward zero (less negative).

14.3 Practical Use - Calculating Pages

```

SELECT COUNT(*) AS total_students,
       CEILING(COUNT(*) / 25) AS pages_needed
FROM students;

```

Output:

```

+-----+-----+
| total_students | pages_needed |
+-----+-----+
|          450   |          18   |
+-----+-----+
1 row in set (0.01 sec)

```

If we display 25 students per page, we need 18 pages to show all 450 students. CEILING ensures we count partial pages.

15. POWER Function

The POWER function raises a number to a specified power (exponent). This is useful for exponential calculations, compound interest, and scientific computations.

15.1 POWER Syntax

```
POWER(base, exponent)
```

15.2 Basic POWER Examples

```
SELECT POWER(2, 3);
```

Output:

```
+-----+
| POWER(2, 3) |
+-----+
|          8  |
+-----+
1 row in set (0.00 sec)
```

This calculates $2^3 = 2 \times 2 \times 2 = 8$.

```
SELECT POWER(5, 2);
```

Output:

```
+-----+
| POWER(5, 2) |
+-----+
```

```

|      25      |
+-----+
1 row in set (0.00 sec)

```

This calculates $5^2 = 5 \times 5 = 25$.

15.3 Fractional Powers (Roots)

```
SELECT POWER(16, 0.5);
```

Output:

```

+-----+
| POWER(16, 0.5) |
+-----+
|      4      |
+-----+
1 row in set (0.00 sec)

```

Raising to the power of 0.5 is the same as taking the square root. $16^{0.5} = \sqrt{16} = 4$.

15.4 Practical Use - Compound Interest

```

SELECT 10000 AS principal,
       10000 * POWER(1.05, 5) AS amount_after_5_years,
       ROUND(10000 * POWER(1.05, 5) - 10000, 2) AS interest_earned;

```

Output:

```

+-----+-----+-----+
| principal | amount_after_5_years | interest_earned |
+-----+-----+-----+

```

10000	12762.815625	2762.82
-------	--------------	---------

1 row in set (0.00 sec)

This calculates compound interest at 5% per year for 5 years using the formula: $A = P(1 + r)^t$.

16. ABS Function

The ABS function returns the absolute value of a number, which is its distance from zero without regard to sign. All negative numbers become positive.

16.1 Basic ABS Examples

```
SELECT ABS(-45);
```

Output:

ABS(-45)
45

1 row in set (0.00 sec)

```
SELECT ABS(45);
```

Output:

```

+-----+
| ABS(45) |
+-----+
|    45   |
+-----+
1 row in set (0.00 sec)

```

Positive numbers remain unchanged.

16.2 Practical Use - Finding Differences

```

SELECT sname, maths, science,
       ABS(maths - science) AS marks_difference
FROM students
ORDER BY marks_difference DESC

```

Output:

```

+-----+-----+-----+-----+
| sname  | maths | science | marks_difference |
+-----+-----+-----+-----+
| Rohan  | 92    | 45      | 47               |
| Aryan  | 38    | 82      | 44               |
| Kavya  | 90    | 48      | 42               |
| Diya   | 41    | 83      | 42               |
| Ishaan | 89    | 49      | 40               |
+-----+-----+-----+-----+
5 rows in set (0.02 sec)

```

This finds students with the largest performance gap between maths and science, regardless of which subject they performed better in.

16.3 Finding Values Within Range

```
SELECT TOP 8 sname, english,  
            ABS(english - 60) AS distance_from_60  
FROM students  
WHERE ABS(english - 60) <= 5;
```

Output:

sname	english	distance_from_60
Priya	58	2
Rahul	62	2
Kavya	57	3
Arnav	63	3
Tanvi	56	4

5 rows in set (0.01 sec)

This finds students whose English marks are within 5 points of 60 (between 55 and 65).

17. MOD Function

The MOD function returns the remainder after dividing one number by another. This is useful for finding even/odd numbers, creating alternating patterns, or checking divisibility.

17.1 MOD Syntax

```
MOD(dividend, divisor)
```

17.2 Basic MOD Examples

```
SELECT MOD(10, 3);
```

Output:

```
+-----+  
| MOD(10, 3) |  
+-----+  
|      1      |  
+-----+  
1 row in set (0.00 sec)
```

10 divided by 3 is 3 with remainder 1.

```
SELECT MOD(17, 5);
```

Output:

```
+-----+  
| MOD(17, 5) |  
+-----+  
|      2      |  
+-----+  
1 row in set (0.00 sec)
```

17 divided by 5 is 3 with remainder 2.

17.3 Finding Even and Odd Numbers

```
SELECT roll, sname,
       CASE WHEN MOD(roll, 2) = 0 THEN 'Even' ELSE 'Odd' END AS roll_type
FROM students
```

Output:

```
+-----+-----+-----+
| roll | sname  | roll_type |
+-----+-----+-----+
| 1    | Aarav  | Odd       |
| 2    | Vivaan | Even      |
| 3    | Aditya | Odd       |
| 4    | Vihaan | Even      |
| 5    | Arjun  | Odd       |
| 6    | Sai    | Even      |
| 7    | Arnav  | Odd       |
| 8    | Krishna| Even      |
+-----+-----+-----+
8 rows in set (0.00 sec)
```

A number is even if MOD(number, 2) equals 0, otherwise it's odd.

17.4 Creating Alternating Groups

```
SELECT TOP 10 roll, sname,
       MOD(roll, 4) AS group_number
FROM students
```

Output:

```
+-----+-----+-----+
| roll | sname  | group_number |
+-----+-----+-----+
| 1    | Aarav  | 1            |
```

2	Vivaan	2
3	Aditya	3
4	Vihaan	0
5	Arjun	1
6	Sai	2
7	Arnav	3
8	Krishna	0
9	Advika	1
10	Ananya	2

```
+-----+-----+-----+
10 rows in set (0.00 sec)
```

This divides students into 4 groups (0-3) based on their roll numbers.

18. Combining Numeric Functions

Numeric functions can be combined to perform complex calculations.

18.1 Calculating Percentage with Rounding

```
SELECT TOP 5 sname,
       english + maths + science AS total_marks,
       ROUND((english + maths + science) * 100 / 300, 2) AS percentage
FROM students
ORDER BY percentage DESC
```

Output:

sname	total_marks	percentage
Arjun	273	91.00
Aarav	268	89.33
Ishaan	265	88.33
Vivaan	261	87.00

```
| Krishna | 258 | 86.00 |
+-----+-----+-----+
5 rows in set (0.02 sec)
```

18.2 Statistical Calculation

```
SELECT TOP 5 sname, maths,
      ROUND(POWER(maths - 60, 2), 2) AS squared_deviation
FROM students
WHERE standard = 10
```

Output:

```
+-----+-----+-----+
| sname | maths | squared_deviation |
+-----+-----+-----+
| Aarav | 85 | 625.00 |
| Vivaan | 72 | 144.00 |
| Aditya | 88 | 784.00 |
| Vihaan | 65 | 25.00 |
| Arjun | 91 | 961.00 |
+-----+-----+-----+
5 rows in set (0.01 sec)
```

This calculates the squared deviation from an average of 60, useful in statistical analysis.

19. Practical Applications

19.1 Data Cleaning Pipeline

```
SELECT
  TOP 5 UPPER(TRIM(sname)) AS clean_name,
  SUBSTRING(address + 1 + 1,
    LOWER(SUBSTRING(address, 2))
  ) AS formatted_address,
  ROUND((english + maths + science) / 3, 1) AS avg_marks
FROM students
WHERE LEN(TRIM(sname)) > 0
```

Output:

clean_name	formatted_address	avg_marks
AARAV	Pune	81.7
VIVAAN	Mumbai	76.3
ADITYA	Delhi	82.0
VIHAAN	Pune	73.3
ARJUN	Chennai	79.7

5 rows in set (0.01 sec)

19.2 Complex Formatting

```
SELECT
  UPPER(SUBSTRING(first_name + 1 + 1),
    LOWER(SUBSTRING(first_name, 2)),
    ' ',
    UPPER(SUBSTRING(last_name, 1, 1)),
    ' ')
```

```
) AS formatted_name,  
'$' + ROUND(esalary + 0) AS formatted_salary  
FROM employees;
```

Output:

```
+-----+-----+  
| formatted_name | formatted_salary |  
+-----+-----+  
| Rajesh K.      | $45000           |  
| Priya S.       | $52500           |  
| Amit P.        | $48750           |  
| Sneha G.       | $31200           |  
+-----+-----+  
4 rows in set (0.00 sec)
```

20. Module Summary

This module covered essential string and numeric functions that enable data transformation and calculation within SQL queries:

String Functions:

- **CONCAT** - Combines multiple strings into one
- **SUBSTRING** - Extracts portions of strings
- **TRIM, LTRIM, RTRIM** - Removes spaces from strings
- **UPPER, LOWER** - Converts case
- **REPLACE** - Substitutes substrings
- **LOCATE** - Finds substring positions
- **LENGTH** - Returns string length

21. SQRT Function

The SQRT function returns the square root of a non-negative number. It is a shorthand alternative to POWER(x, 0.5) and is commonly used in geometry, statistics (standard deviation), and physics calculations.

21.1 SQRT Syntax

```
SQRT(number)
```

21.2 Basic SQRT Examples

```
SELECT SQRT(25);
```

Output:

```
+-----+
| SQRT(25) |
+-----+
|      5   |
+-----+
1 row in set (0.00 sec)
```

```
SELECT SQRT(2);
```

Output:

```
+-----+
| SQRT(2) |
+-----+
```

```
| 1.4142135623730951 |
+-----+
1 row in set (0.00 sec)
```

The square root of 2 is irrational — SQL Server returns it as a floating-point number.

21.3 SQRT with Rounding

```
SELECT ROUND(SQRT(2), 4);
```

Output:

```
+-----+
| ROUND(SQRT(2), 4) |
+-----+
|          1.4142    |
+-----+
1 row in set (0.00 sec)
```

21.4 Practical Use — Distance Between Two Points

SQRT is essential in the Euclidean distance formula: $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$

```
SELECT student_a, student_b,
       ROUND(SQRT(POWER(maths_b - maths_a, 2) + POWER(science_b - science_a, 2)), 2) AS
score_distance
FROM (
  SELECT 'Aarav' AS student_a, 85 AS maths_a, 82 AS science_a,
         'Arjun' AS student_b, 91 AS maths_b, 88 AS science_b
) AS scores;
```

Output:

```

+-----+-----+-----+
| student_a | student_b | score_distance |
+-----+-----+-----+
| Aarav      | Arjun      | 8.49           |
+-----+-----+-----+
1 row in set (0.00 sec)

```

This calculates how "far apart" two students are in terms of their combined maths and science performance.

21.5 NULL Handling

```
SELECT SQRT(-4);
```

Output:

```

+-----+
| SQRT(-4) |
+-----+
| NULL      |
+-----+
1 row in set (0.00 sec)

```

SQL Server returns NULL for negative inputs (no real square root exists). Always validate input with ABS or a CASE check when data may contain negatives.

22. SQUARE Function

SQUARE is a SQL Server-specific shorthand for POWER(x, 2). It returns the square of a number. While POWER(x, 2) works in all databases, SQUARE is cleaner and more readable for squaring operations.

22.1 SQUARE Syntax

```
SQUARE(number)
```

22.2 Basic SQUARE Examples

```
SELECT SQUARE(5);
```

Output:

```
+-----+  
| SQUARE(5) |  
+-----+  
|      25      |  
+-----+  
1 row in set (0.00 sec)
```

```
SELECT SQUARE(12);
```

Output:

```
+-----+  
| SQUARE(12) |  
+-----+  
|      144      |  
+-----+  
1 row in set (0.00 sec)
```

22.3 Practical Use — Calculating Variance Contributions

```
SELECT TOP 5 sname, maths,  
            75 AS class_avg,  
            SQUARE(maths - 75) AS squared_deviation  
FROM students  
ORDER BY squared_deviation DESC;
```

Output:

sname	maths	class_avg	squared_deviation
Arjun	91	75	256
Aditya	88	75	169
Aarav	85	75	100
Vivaan	72	75	9
Vihaan	65	75	100

5 rows in set (0.01 sec)

Squared deviations are the building block of variance and standard deviation — showing how spread out each student's marks are from the class average.

23. SIGN Function

The SIGN function returns -1, 0, or 1 depending on whether a number is negative, zero, or positive. It is useful for conditional logic, separating gains from losses, or normalizing direction values.

23.1 SIGN Syntax

```
SIGN(number)
```

23.2 Basic SIGN Examples

```
SELECT SIGN(-50), SIGN(0), SIGN(75);
```

Output:

```
+-----+-----+-----+
| SIGN(-50) | SIGN(0) | SIGN(75) |
+-----+-----+-----+
|    -1    |    0    |    1    |
+-----+-----+-----+
1 row in set (0.00 sec)
```

23.3 Practical Use — Classifying Score Changes

```
SELECT sname,
       maths      AS current_marks,
       prev_maths AS previous_marks,
       maths - prev_maths AS change,
       CASE SIGN(maths - prev_maths)
         WHEN 1 THEN 'Improved'
         WHEN 0 THEN 'No Change'
         WHEN -1 THEN 'Declined'
       END AS trend
FROM (
  SELECT 'Aarav' AS sname, 85 AS maths, 78 AS prev_maths UNION ALL
  SELECT 'Vivaan',      72,      72      UNION ALL
  SELECT 'Aditya',      88,      91      UNION ALL
  SELECT 'Vihaan',      65,      60      UNION ALL
  SELECT 'Arjun',      91,      85
) AS scores;
```

Output:

```

+-----+-----+-----+-----+-----+
| sname | current_marks | previous_marks | change | trend |
+-----+-----+-----+-----+-----+
| Aarav | 85 | 78 | 7 | Improved |
| Vivaan | 72 | 72 | 0 | No Change |
| Aditya | 88 | 91 | -3 | Declined |
| Vihaan | 65 | 60 | 5 | Improved |
| Arjun | 91 | 85 | 6 | Improved |
+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)

```

SIGN combined with CASE gives clean, readable trend labels without complex IF/ELSE logic.

24. LOG and LOG10 Functions

Logarithm functions are the inverse of exponentiation. LOG returns the natural logarithm (base $e \approx 2.71828$), while LOG10 returns the base-10 logarithm. These are used in financial modelling, information theory, and scaling large numeric ranges.

24.1 LOG Syntax

```

LOG(number)           -- natural log (ln)
LOG(number, base)     -- log with custom base (SQL Server supports this)
LOG10(number)         -- base-10 log

```

24.2 Basic LOG and LOG10 Examples

```

SELECT LOG(1),
       LOG(2.71828182845905),
       LOG10(100),
       LOG10(1000);

```

Output:

```

+-----+-----+-----+-----+
| LOG(1) | LOG(2.71828182845905) | LOG10(100) | LOG10(1000) |
+-----+-----+-----+-----+
| 0      | 0.9999999999999998    | 2          | 3          |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

$\text{LOG}(1) = 0$ because $e^0 = 1$. $\text{LOG10}(100) = 2$ because $10^2 = 100$.

24.3 Custom Base Logarithm

```

-- Log base 2 of 8 (2^3 = 8, so answer should be 3)
SELECT LOG(8, 2) AS log_base2_of_8;

```

Output:

```

+-----+
| log_base2_of_8 |
+-----+
| 3.0            |
+-----+
1 row in set (0.00 sec)

```

24.4 Practical Use — Scaling Salary Data

```

SELECT TOP 5 first_name + ' ' + last_name AS emp_name,
       esalary,
       ROUND(LOG10(esalary), 4) AS log_salary
FROM employees
ORDER BY esalary DESC;

```

Output:

```
+-----+-----+-----+
| emp_name | esalary | log_salary |
+-----+-----+-----+
| Priya Sharma | 52500.00 | 4.7202 |
| Amit Patel | 48750.00 | 4.6879 |
| Rajesh Kumar | 45000.00 | 4.6532 |
| Sneha Gupta | 31200.00 | 4.4942 |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Log-scaling compresses large numeric ranges — useful when visualising salaries, populations, or scores that span several orders of magnitude.

25. EXP Function

The EXP function returns e raised to the power of the given number (e^x). It is the inverse of LOG. EXP is used in exponential growth/decay models, probability calculations, and undoing natural log transformations.

25.1 EXP Syntax

```
EXP(number)
```

25.2 Basic EXP Examples

```
SELECT EXP(0),
       EXP(1),
       ROUND(EXP(2), 4);
```

Output:

```

+-----+-----+-----+
| EXP(0) | EXP(1)          | ROUND(EXP(2), 4) |
+-----+-----+-----+
| 1      | 2.718281828459045 | 7.3891           |
+-----+-----+-----+
1 row in set (0.00 sec)

```

$\text{EXP}(0) = 1$, $\text{EXP}(1) = e$ (Euler's number ≈ 2.71828).

25.3 LOG and EXP Are Inverses

```
SELECT EXP(LOG(50)) AS round_trip;
```

Output:

```

+-----+
| round_trip |
+-----+
| 50.0       |
+-----+
1 row in set (0.00 sec)

```

Applying EXP to a LOG (or LOG to an EXP) returns the original number — they cancel each other out.

25.4 Practical Use — Exponential Growth Model

```

-- Predicted student enrollment after N years
-- Formula: initial_count * e^(growth_rate * years)
SELECT 1000 AS initial_enrollment,
       0.05 AS annual_growth_rate,

```

```

year_num,
ROUND(1000 * EXP(0.05 * year_num), 0) AS predicted_enrollment
FROM (VALUES (1),(2),(3),(5),(10)) AS t(year_num);

```

Output:

initial_enrollment	annual_growth_rate	year_num	predicted_enrollment
1000	0.05	1	1051
1000	0.05	2	1105
1000	0.05	3	1162
1000	0.05	5	1284
1000	0.05	10	1649

5 rows in set (0.01 sec)

26. PI Function

PI returns the mathematical constant π (approximately 3.14159265358979). It is used in geometry calculations such as circumference and area of circles.

26.1 PI Syntax

```
PI()
```

26.2 Basic PI Example

```
SELECT PI();
```

Output:


```

+-----+
| PI()   |
+-----+
| 3.14159265358979 |
+-----+
1 row in set (0.00 sec)

```

26.3 Practical Use — Circle Geometry

```

SELECT shape,
       radius,
       ROUND(PI() * radius * radius, 2) AS area,
       ROUND(2 * PI() * radius, 2) AS circumference
FROM (VALUES
      ('Small', 5),
      ('Medium', 10),
      ('Large', 20)
) AS circles(shape, radius);

```

Output:

```

+-----+-----+-----+-----+
| shape | radius | area  | circumference |
+-----+-----+-----+-----+
| Small | 5      | 78.54 | 31.42         |
| Medium | 10     | 314.16 | 62.83         |
| Large  | 20     | 1256.64 | 125.66        |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

27. RAND Function

RAND returns a random floating-point number between 0 (inclusive) and 1 (exclusive). An optional seed value can be provided to generate a reproducible "random" sequence — useful for testing.

27.1 RAND Syntax

```
RAND()          -- different result each call
RAND(seed)      -- reproducible result for a given seed
```

27.2 Basic RAND Examples

```
SELECT RAND();
```

Output:

```
+-----+
| RAND() |
+-----+
| 0.7183421634892 |
+-----+
1 row in set (0.00 sec)
```

Your result will differ — RAND() returns a different value each time.

```
SELECT RAND(42);
```

Output:

```
+-----+
| RAND(42) |
+-----+
```

```
| 0.74404503552 |
+-----+
1 row in set (0.00 sec)
```

Using seed 42 always returns the same value — useful for repeatable test scenarios.

27.3 Generating Random Integers in a Range

To get a random integer between `low` and `high` (inclusive):

```
-- Random integer between 1 and 100
SELECT FLOOR(RAND() * 100) + 1 AS random_score;
```

Output:

```
+-----+
| random_score |
+-----+
|          67  |
+-----+
1 row in set (0.00 sec)
```

27.4 Practical Use — Randomly Sampling Students

```
SELECT TOP 5 sname, maths
FROM students
ORDER BY NEWID(); -- NEWID() generates a random GUID, giving a random row order
```

Output:

```
+-----+-----+
| sname  | maths |
+-----+-----+
```

```

| Kavya | 90 |
| Priya | 77 |
| Rohan | 92 |
| Diya  | 41 |
| Arnav | 63 |
+-----+
5 rows in set (0.02 sec)

```

Note: In SQL Server, `ORDER BY NEWID()` is the preferred way to randomly shuffle rows. `ORDER BY RAND()` only evaluates once per query and does not shuffle rows reliably in SQL Server.

28. TRUNCATE / Integer Division in SQL Server

MySQL has a dedicated `TRUNCATE()` function for numbers. SQL Server does not — instead, you use `ROUND(number, decimal_places, 1)` where the third argument `1` means "truncate instead of round", or cast to an integer type.

28.1 Truncating Decimals in SQL Server

```

-- Truncate to 2 decimal places (no rounding)
SELECT ROUND(123.789, 2, 1) AS truncated;

```

Output:

```

+-----+
| truncated |
+-----+
| 123.78   |
+-----+
1 row in set (0.00 sec)

```

```
-- Compare ROUND vs TRUNCATE behaviour
SELECT ROUND(2.999, 0)          AS rounded,
       ROUND(2.999, 0, 1)      AS truncated;
```

Output:

```
+-----+-----+
| rounded | truncated |
+-----+-----+
|      3 |        2 |
+-----+-----+
1 row in set (0.00 sec)
```

ROUND(2.999, 0) rounds up to 3. ROUND(2.999, 0, 1) truncates to 2 — it simply drops the decimal part without considering whether to round up.

28.2 Truncating to Integer with CAST

```
SELECT CAST(123.789 AS INT) AS truncated_to_int,
       CAST(-45.9 AS INT) AS negative_truncated;
```

Output:

```
+-----+-----+
| truncated_to_int | negative_truncated |
+-----+-----+
|          123     |          -45       |
+-----+-----+
1 row in set (0.00 sec)
```

CAST to INT always truncates toward zero — it does not round, and works on both positive and negative values.

28.3 Practical Use — Extracting Whole Units

```
SELECT TOP 5 sname,
      (english + maths + science) / 3.0 AS exact_avg,
      ROUND((english + maths + science) / 3.0, 0, 1) AS truncated_avg,
      ROUND((english + maths + science) / 3.0, 0) AS rounded_avg
FROM students
ORDER BY exact_avg DESC;
```

Output:

```
+-----+-----+-----+-----+
| sname | exact_avg | truncated_avg | rounded_avg |
+-----+-----+-----+-----+
| Anjun | 91.00 | 91 | 91 |
| Aarav | 89.33 | 89 | 89 |
| Ishaan | 88.33 | 88 | 88 |
| Vivaan | 87.00 | 87 | 87 |
| Aditya | 82.00 | 82 | 82 |
+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

28A. Cross-Database Comparison: Text/String Functions

String manipulation syntax varies significantly across database systems. The table below shows how each text function is implemented in SQL Server (T-SQL), MySQL, PostgreSQL, and Oracle. Where syntax differs, the exact equivalent is shown. A ✓ means the function name and syntax are identical to SQL Server.

Function / Purpose	SQL Server (T-SQL)	MySQL	PostgreSQL	Oracle
String concatenation	'Hello' + ' ' + 'World'	CONCAT('Hello', ' ', 'World')	'Hello' ' ' 'World' or CONCAT(...)	'Hello' ' ' 'World'

Function / Purpose	SQL Server (T-SQL)	MySQL	PostgreSQL	Oracle
Extract substring	<code>SUBSTRING(str, start, len)</code>	<code>SUBSTRING(str, start, len)</code> or <code>SUBSTR(...)</code>	✓ <code>SUBSTRING(str, start, len)</code> or <code>SUBSTR(...)</code>	<code>SUBSTR(str, start, len)</code>
Extract from position to end	<code>SUBSTRING(str, start)</code>	✓ <code>SUBSTRING(str, start)</code>	✓ <code>SUBSTRING(str, start)</code>	<code>SUBSTR(str, start)</code>
Trim spaces (both ends)	<code>TRIM(str)</code>	✓ <code>TRIM(str)</code>	✓ <code>TRIM(str)</code>	✓ <code>TRIM(str)</code>
Trim left/leading spaces	<code>LTRIM(str)</code>	✓ <code>LTRIM(str)</code>	✓ <code>LTRIM(str)</code>	✓ <code>LTRIM(str)</code>
Trim right/trailing spaces	<code>RTRIM(str)</code>	✓ <code>RTRIM(str)</code>	✓ <code>RTRIM(str)</code>	✓ <code>RTRIM(str)</code>
Convert to uppercase	<code>UPPER(str)</code>	✓ <code>UPPER(str)</code>	✓ <code>UPPER(str)</code>	✓ <code>UPPER(str)</code>
Convert to lowercase	<code>LOWER(str)</code>	✓ <code>LOWER(str)</code>	✓ <code>LOWER(str)</code>	✓ <code>LOWER(str)</code>
Replace substring	<code>REPLACE(str, old, new)</code>	✓ <code>REPLACE(str, old, new)</code>	✓ <code>REPLACE(str, old, new)</code>	✓ <code>REPLACE(str, old, new)</code>
Find substring position	<code>CHARINDEX(substr, str)</code>	<code>LOCATE(substr, str)</code> or <code>INSTR(str, substr)</code>	<code>POSITION(substr IN str)</code> or <code>STRPOS(str, substr)</code>	<code>INSTR(str, substr)</code>
Find substring (with start position)	<code>CHARINDEX(substr, str, start)</code>	<code>LOCATE(substr, str, start)</code>	<code>STRPOS(SUBSTRING(str FROM start), substr) + start - 1</code>	<code>INSTR(str, substr, start)</code>

Function / Purpose	SQL Server (T-SQL)	MySQL	PostgreSQL	Oracle
String length	LEN(str)	LENGTH(str) or CHAR_LENGTH(str)	LENGTH(str) or CHAR_LENGTH(str)	LENGTH(str)
Left n characters	LEFT(str, n)	✓ LEFT(str, n)	✓ LEFT(str, n)	SUBSTR(str, 1, n)
Right n characters	RIGHT(str, n)	✓ RIGHT(str, n)	✓ RIGHT(str, n)	SUBSTR(str, -n)
Repeat string n times	REPLICATE(str, n)	REPEAT(str, n)	REPEAT(str, n)	RPAD(' ', n*LENGTH(str), str)
Reverse string	REVERSE(str)	✓ REVERSE(str)	✓ REVERSE(str)	✓ REVERSE(str)
Pad left with character	REPLICATE(char, n) + str	LPAD(str, total_len, pad_char)	LPAD(str, total_len, pad_char)	LPAD(str, total_len, pad_char)
Pad right with character	str + REPLICATE(char, n)	RPAD(str, total_len, pad_char)	RPAD(str, total_len, pad_char)	RPAD(str, total_len, pad_char)
Remove leading/trailing chars	No direct function — use TRIM() + REPLACE()	TRIM(BOTH 'x' FROM str)	TRIM(BOTH 'x' FROM str)	TRIM(BOTH 'x' FROM str)
ASCII value of first character	ASCII(str)	✓ ASCII(str)	✓ ASCII(str)	✓ ASCII(str)
Character from ASCII value	CHAR(n)	✓ CHAR(n)	CHR(n)	CHR(n)

Function / Purpose	SQL Server (T-SQL)	MySQL	PostgreSQL	Oracle
String comparison (case-insensitive)	WHERE LOWER(col) = LOWER(val)	WHERE col COLLATE utf8mb4_general _ci = val	WHERE LOWER(col) = LOWER(val) or use ILIKE	WHERE UPPER(col) = UPPER(val)
Format number as string	FORMAT(num, format) e.g., FORMAT(1234.5, 'N2')	FORMAT(num, decimal_places)	TO_CHAR(num, format)	TO_CHAR(num, format)
Convert string to number	CAST(str AS DECIMAL) or CONVERT(DECIMAL , str)	CAST(str AS DECIMAL)	CAST(str AS NUMERIC) or ::numeric	TO_NUMBER(str)
Convert string to date	CAST(str AS DATE) or CONVERT(DATE, str)	STR_TO_DATE(str , format)	TO_DATE(str, format)	TO_DATE(str, format)
Soundex (phonetic matching)	SOUNDEX(str)	✓ SOUNDEX(str)	Enable fuzzystrmatch extension, then SOUNDEX(str)	✓ SOUNDEX(str)
Difference (phonetic similarity 0-4)	DIFFERENCE(str1 , str2)	No direct equivalent	Use LEVENSHTEIN() from fuzzystrmatch	No direct equivalent

⚠ **Key gotcha — Concatenation:** SQL Server uses the `+` operator for string concatenation. MySQL uses `CONCAT()` function. PostgreSQL and Oracle use the `||` operator. Be careful when porting queries between systems.

⚠ **Key gotcha — Position functions:** SQL Server uses `CHARINDEX(needle, haystack)`. MySQL uses `LOCATE(needle, haystack)` or `INSTR(haystack, needle)` (argument order reversed!). PostgreSQL uses `POSITION(needle IN haystack)`. Oracle uses `INSTR(haystack, needle)`.

⚠ **Key gotcha — Length functions:** SQL Server uses `LEN()` which ignores trailing spaces. MySQL, PostgreSQL, and Oracle use `LENGTH()` which counts trailing spaces. For consistent behavior across platforms, use `DATALENGTH()` in SQL Server or ensure strings are trimmed.

⚠ **Key gotcha — SUBSTRING vs SUBSTR:** SQL Server and MySQL primarily use `SUBSTRING()`. Oracle uses `SUBSTR()`. PostgreSQL supports both. The function signatures are the same: `(string, start_position, length)`.

29. Cross-Database Comparison: Numeric Functions

The table below summarises how each numeric function is written across SQL Server (T-SQL), MySQL, PostgreSQL, and Oracle. Where syntax differs, the exact equivalent is shown. A ✓ means the function name and syntax are identical to SQL Server.

Function / Purpose	SQL Server (T-SQL)	MySQL	PostgreSQL	Oracle
Round a number	<code>ROUND(x, n)</code>	✓ <code>ROUND(x, n)</code>	✓ <code>ROUND(x, n)</code>	✓ <code>ROUND(x, n)</code>
Round down (floor)	<code>FLOOR(x)</code>	✓ <code>FLOOR(x)</code>	✓ <code>FLOOR(x)</code>	✓ <code>FLOOR(x)</code>
Round up (ceiling)	<code>CEILING(x)</code>	<code>CEILING(x)</code> or <code>CEIL(x)</code>	<code>CEILING(x)</code> or <code>CEIL(x)</code>	<code>CEIL(x)</code> — <code>CEILING</code> not supported
Raise to a power	<code>POWER(x, n)</code>	✓ <code>POWER(x, n)</code>	<code>POWER(x, n)</code> or <code>x ^ n</code>	✓ <code>POWER(x, n)</code>
Absolute value	<code>ABS(x)</code>	✓ <code>ABS(x)</code>	✓ <code>ABS(x)</code>	✓ <code>ABS(x)</code>

Function / Purpose	SQL Server (T-SQL)	MySQL	PostgreSQL	Oracle
Modulus / remainder	<code>x % y</code> or <code>x % y</code>	<code>MOD(x, y)</code> or <code>x % y</code>	<code>MOD(x, y)</code> or <code>x % y</code>	<code>MOD(x, y)</code> — <code>%</code> operator not supported
Square root	<code>SQRT(x)</code>	✓ <code>SQRT(x)</code>	✓ <code>SQRT(x)</code>	✓ <code>SQRT(x)</code>
Square of a number	<code>SQUARE(x)</code>	No equivalent — use <code>POWER(x, 2)</code>	No equivalent — use <code>POWER(x, 2)</code>	No equivalent — use <code>POWER(x, 2)</code>
Sign of a number	<code>SIGN(x)</code>	✓ <code>SIGN(x)</code>	✓ <code>SIGN(x)</code>	✓ <code>SIGN(x)</code>
Natural log (ln)	<code>LOG(x)</code>	<code>LOG(x)</code> or <code>LN(x)</code>	<code>LN(x)</code> — <code>LOG(x)</code> is base-10 in PostgreSQL!	<code>LN(x)</code> — <code>LOG(x)</code> is base-10 in Oracle!
Base-10 log	<code>LOG10(x)</code>	✓ <code>LOG10(x)</code>	<code>LOG(x)</code> or <code>LOG10(x)</code>	<code>LOG(10, x)</code> or <code>LOG(x)</code>
Custom base log	<code>LOG(x, base)</code>	<code>LOG(base, x)</code> — argument order reversed!	<code>LOG(base, x)</code> — argument order reversed!	<code>LOG(base, x)</code> — argument order reversed!
Exponential (e^x)	<code>EXP(x)</code>	✓ <code>EXP(x)</code>	✓ <code>EXP(x)</code>	✓ <code>EXP(x)</code>
Pi constant	<code>PI()</code>	✓ <code>PI()</code>	<code>PI()</code> or literal <code>3.14159265358979</code>	No <code>PI()</code> function — use <code>ACOS(-1)</code> to get π
Random number (0–1)	<code>RAND()</code>	✓ <code>RAND()</code>	<code>RANDOM()</code>	<code>DBMS_RANDOM.VALUE</code>

Function / Purpose	SQL Server (T-SQL)	MySQL	PostgreSQL	Oracle
Random row ordering	<code>ORDER BY NEWID()</code>	<code>ORDER BY RAND()</code>	<code>ORDER BY RANDOM()</code>	<code>ORDER BY DBMS_RANDOM.VALUE</code>
Truncate decimals	<code>ROUND(x, n, 1)</code> (3rd arg = 1)	<code>TRUNCATE(x, n)</code>	<code>TRUNC(x, n)</code>	<code>TRUNC(x, n)</code>
Truncate to integer	<code>CAST(x AS INT)</code>	<code>TRUNCATE(x, 0)</code>	<code>TRUNC(x)</code>	<code>TRUNC(x)</code>

⚠ **Key gotcha — LOG base order:** SQL Server uses `LOG(x, base)` while MySQL, PostgreSQL, and Oracle use `LOG(base, x)`. The arguments are *reversed*. Always double-check this when porting queries.

⚠ **Key gotcha — LOG vs LN:** In SQL Server, `LOG(x)` is the *natural* log. In PostgreSQL and Oracle, `LOG(x)` is the *base-10* log. Use `LN(x)` in PostgreSQL/Oracle for natural log.

20. Module Summary

This module covered essential string and numeric functions that enable data transformation and calculation within SQL queries:

String Functions:

- **+** (Concatenation) - Combines multiple strings into one (SQL Server uses `+`; MySQL/PostgreSQL use `CONCAT()`)
- **SUBSTRING** - Extracts portions of strings
- **TRIM, LTRIM, RTRIM** - Removes spaces from strings
- **UPPER, LOWER** - Converts case
- **REPLACE** - Substitutes substrings
- **CHARINDEX** - Finds substring positions (equivalent to MySQL's `LOCATE`)

- **LEN** - Returns string length (equivalent to MySQL/PostgreSQL's `LENGTH`)

Numeric Functions:

- **ROUND** - Rounds a number to specified decimal places
- **FLOOR** - Always rounds down to the nearest integer
- **CEILING** - Always rounds up to the nearest integer
- **POWER** - Raises a number to a specified exponent
- **ABS** - Returns absolute value (removes negative sign)
- **% operator** - Returns the remainder (modulus) of division
- **SQRT** - Returns the square root of a number
- **SQUARE** - Returns the square of a number (SQL Server only)
- **SIGN** - Returns -1, 0, or 1 indicating sign of a number
- **LOG / LOG10** - Natural and base-10 logarithms
- **EXP** - Returns e raised to the given power
- **PI** - Returns the constant π
- **RAND / NEWID** - Generates random values and random row ordering
- **ROUND(x, n, 1)** - Truncates (instead of rounding) to n decimal places

These functions form the foundation for data cleaning, formatting, and mathematical analysis in SQL. They can be combined and nested to perform complex transformations within a single query.